



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 07: Emergence (cap 12)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

26 de Agosto, 2025

Proyecto

Shin Megami Tensei

Shin Megami Tensei es una serie de videojuegos de rol japones desarrollada por Atlus. Cuenta con más de 30 años de historia, pero suele vivir bajo la sombra de su serie hermana Persona.



Sistema de turnos

			
1317 HP	137 HP	133 HP	
625 MP	131 MP	58 MP	
Shujinko	Pyro Jack	Jack Ripper	
			
1	4	3	2
597 HP	103 HP	388 HP	445 HP
347 MP	113 MP	76 MP	85 MP
Makoto	Jack Frost	Black Frost	King Frost



Sistema de turnos

 1317 HP 625 MP Shujinko	 99 HP 131 MP Pyro Jack	 133 HP 58 MP Jack Ripper	
 597 HP 347 MP Makoto	 103 HP 113 MP Jack Frost	 388 HP 76 MP Black Frost	 445 HP 85 MP King Frost



Sistema de turnos

 1317 HP 625 MP Shujinko	 137 HP 131 MP Pyro Jack	 116 HP 58 MP Jack Ripper	
 597 HP 347 MP Makoto	 103 HP 113 MP Jack Frost	 388 HP 76 MP Black Frost	 445 HP 85 MP King Frost

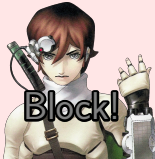


Sistema de turnos

 Weak! 57 HP DAMAGE			
1260 HP	137 HP	133 HP	
625 MP	131 MP	58 MP	
Shujinko	Pyro Jack	Jack Ripper	
 1	 4	 3	 2
597 HP	103 HP	388 HP	445 HP
347 MP	113 MP	76 MP	85 MP
Makoto	Jack Frost	Black Frost	King Frost

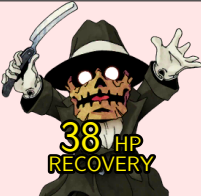


Sistema de turnos

 Block!			
1317 HP	137 HP	133 HP	
625 MP	131 MP	58 MP	
Shujinko	Pyro Jack	Jack Ripper	
 1	 4	 3	 2
597 HP	103 HP	388 HP	445 HP
347 MP	113 MP	76 MP	85 MP
Makoto	Jack Frost	Black Frost	King Frost



Sistema de turnos

		 38 HP RECOVERY	
1317 HP 625 MP	137 HP 131 MP	133 HP 58 MP	
Shujinko	Pyro Jack	Jack Ripper	
 1	 4	 3	 2
597 HP 347 MP	103 HP 113 MP	388 HP 76 MP	445 HP 85 MP
Makoto	Jack Frost	Black Frost	King Frost

Sistema de turnos

			
1317 HP	137 HP	133 HP	
625 MP	131 MP	58 MP	
Shujinko	Pyro Jack	Jack Ripper	
			
1	4	3	2
559 HP	103 HP	388 HP	445 HP
347 MP	113 MP	76 MP	85 MP
Makoto	Jack Frost	Black Frost	King Frost

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y el consumo de turnos de los jugadores. En particular, el proyecto cuenta con los siguientes desafíos:

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y el consumo de turnos de los jugadores. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las unidades tienen distintas afinidades a los diferentes elementos

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y el consumo de turnos de los jugadores. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las unidades tienen distintas afinidades a los diferentes elementos
- 2 Las afinidades tienen distintos efectos en los turnos

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y el consumo de turnos de los jugadores. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las unidades tienen distintas afinidades a los diferentes elementos
- 2 Las afinidades tienen distintos efectos en los turnos
- 3 Existen más de 5 tipos de habilidades

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y el consumo de turnos de los jugadores. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las unidades tienen distintas afinidades a los diferentes elementos
- 2 Las afinidades tienen distintos efectos en los turnos
- 3 Existen más de 5 tipos de habilidades
- 4 Todos los tipos de habilidades hacen cosas muy distintas

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y el consumo de turnos de los jugadores. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las unidades tienen distintas afinidades a los diferentes elementos
- 2 Las afinidades tienen distintos efectos en los turnos
- 3 Existen más de 5 tipos de habilidades
- 4 Todos los tipos de habilidades hacen cosas muy distintas

Pregunta importante: ¿Como diseñamos nuestro código para que agregar nuevos tipos de habilidades sea simple?

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de turnos, algunas acciones adicionales y dos tipos de habilidades. En particular:

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de turnos, algunas acciones adicionales y dos tipos de habilidades. En particular:

- 1 Se deben implementar todas las afinidades del juego

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de turnos, algunas acciones adicionales y dos tipos de habilidades. En particular:

- 1 Se deben implementar todas las afinidades del juego
- 2 Deben implementar las acciones Invocar y Pasar Turno

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de turnos, algunas acciones adicionales y dos tipos de habilidades. En particular:

- 1 Se deben implementar todas las afinidades del juego
- 2 Deben implementar las acciones Invocar y Pasar Turno
- 3 Deben implementar todas las habilidades ofensivas que tienen un solo rival como objetivo

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de turnos, algunas acciones adicionales y dos tipos de habilidades. En particular:

- 1 Se deben implementar todas las afinidades del juego
- 2 Deben implementar las acciones Invocar y Pasar Turno
- 3 Deben implementar todas las habilidades ofensivas que tienen un solo rival como objetivo
- 4 Deben implementar todas las habilidades de curación con un solo objetivo

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de turnos, algunas acciones adicionales y dos tipos de habilidades. En particular:

- 1 Se deben implementar todas las afinidades del juego
- 2 Deben implementar las acciones Invocar y Pasar Turno
- 3 Deben implementar todas las habilidades ofensivas que tienen un solo rival como objetivo
- 4 Deben implementar todas las habilidades de curación con un solo objetivo

Los principales desafíos serán diseñar el sistema de afinidades y las habilidades de tal forma que implementar nuevos elementos sea sencillo.

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de turnos, algunas acciones adicionales y dos tipos de habilidades. En particular:

- 1 Se deben implementar todas las afinidades del juego
- 2 Deben implementar las acciones Invocar y Pasar Turno
- 3 Deben implementar todas las habilidades ofensivas que tienen un solo rival como objetivo
- 4 Deben implementar todas las habilidades de curación con un solo objetivo

Los principales desafíos serán diseñar el sistema de afinidades y las habilidades de tal forma que implementar nuevos elementos sea sencillo.

En esta entrega, evaluaremos los capítulos 2, 3, 6 y 10 de Clean Code, así como el patrón Modelo-Vista-Controlador.

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un objetivo, habilidades de muerte instantánea y un nuevo tipo de habilidades. En particular:

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un objetivo, habilidades de muerte instantánea y un nuevo tipo de habilidades. En particular:

- 1 Deben modificar el sistema de turnos para incluir ataques con múltiples objetivos

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un objetivo, habilidades de muerte instantánea y un nuevo tipo de habilidades. En particular:

- 1 Deben modificar el sistema de turnos para incluir ataques con múltiples objetivos
- 2 Deben agregar las condiciones de muerte instantánea para cada afinidad

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un objetivo, habilidades de muerte instantánea y un nuevo tipo de habilidades. En particular:

- 1 Deben modificar el sistema de turnos para incluir ataques con múltiples objetivos
- 2 Deben agregar las condiciones de muerte instantánea para cada afinidad
- 3 Deben implementar ataques del elemento **Almighty**

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un objetivo, habilidades de muerte instantánea y un nuevo tipo de habilidades. En particular:

- 1 Deben modificar el sistema de turnos para incluir ataques con múltiples objetivos
- 2 Deben agregar las condiciones de muerte instantánea para cada afinidad
- 3 Deben implementar ataques del elemento **Almighty**
- 4 Deben implementar una interfaz gráfica que nosotros les entregaremos

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un objetivo, habilidades de muerte instantánea y un nuevo tipo de habilidades. En particular:

- 1 Deben modificar el sistema de turnos para incluir ataques con múltiples objetivos
- 2 Deben agregar las condiciones de muerte instantánea para cada afinidad
- 3 Deben implementar ataques del elemento **Almighty**
- 4 Deben implementar una interfaz gráfica que nosotros les entregaremos

Los principales desafíos serán extender el sistema de afinidades y crear las nuevas habilidades sin repetir código

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un objetivo, habilidades de muerte instantánea y un nuevo tipo de habilidades. En particular:

- 1 Deben modificar el sistema de turnos para incluir ataques con múltiples objetivos
- 2 Deben agregar las condiciones de muerte instantánea para cada afinidad
- 3 Deben implementar ataques del elemento **Almighty**
- 4 Deben implementar una interfaz gráfica que nosotros les entregaremos

Los principales desafíos serán extender el sistema de afinidades y crear las nuevas habilidades sin repetir código

Se pondrá a prueba que sus códigos permitan cambiar de una vista de consola a una vista de interfaz gráfica de forma sencilla y poco disruptiva.

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un objetivo, habilidades de muerte instantánea y un nuevo tipo de habilidades. En particular:

- 1 Deben modificar el sistema de turnos para incluir ataques con múltiples objetivos
- 2 Deben agregar las condiciones de muerte instantánea para cada afinidad
- 3 Deben implementar ataques del elemento **Almighty**
- 4 Deben implementar una interfaz gráfica que nosotros les entregaremos

Los principales desafíos serán extender el sistema de afinidades y crear las nuevas habilidades sin repetir código

Se pondrá a prueba que sus códigos permitan cambiar de una vista de consola a una vista de interfaz gráfica de forma sencilla y poco disruptiva.

En esta entrega, evaluaremos todos los capítulos de Clean Code (que vemos en clases) y el patrón Modelo-Vista-Controlador.

Esta entrega es opcional y está pensada como una “*E3 en esteroides*”. En particular:

Esta entrega es opcional y está pensada como una “*E3 en esteroides*”. En particular:

- 1 Deben implementar las habilidades Support, las que añaden efectos adicionales al juego

Esta entrega es opcional y está pensada como una “*E3 en esteroides*”. En particular:

- 1 Deben implementar las habilidades Support, las que añaden efectos adicionales al juego
- 2 Deben implementar habilidades con efectos especiales

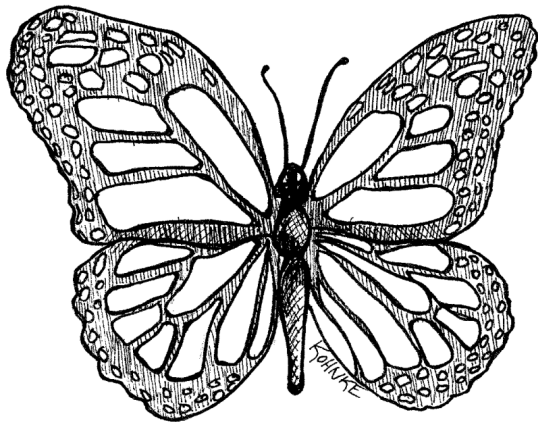
Esta entrega es opcional y está pensada como una “*E3 en esteroides*”. En particular:

- 1 Deben implementar las habilidades Support, las que añaden efectos adicionales al juego
- 2 Deben implementar habilidades con efectos especiales

Esta entrega será la última instancia para demostrar que pueden ganarle a los desafíos del proyecto.

Emergence

by Jeff Langr



Todos queremos generar software de calidad, con diseños robustos que nos permitan agregarle funcionalidades y adaptarnos a los cambios pedidos por el usuario.

Todos queremos generar software de calidad, con diseños robustos que nos permitan agregarle funcionalidades y adaptarnos a los cambios pedidos por el usuario.

¿Pero cómo lo logramos?

Todos queremos generar software de calidad, con diseños robustos que nos permitan agregarle funcionalidades y adaptarnos a los cambios pedidos por el usuario.

¿Pero cómo lo logramos?

Idea: Antes de programar, dediquemos mucho tiempo a crear un diseño perfecto

Todos queremos generar software de calidad, con diseños robustos que nos permitan agregarle funcionalidades y adaptarnos a los cambios pedidos por el usuario.

¿Pero cómo lo logramos?

Idea: Antes de programar, dediquemos mucho tiempo a crear un diseño perfecto

¿Alguien ve algún problema con esta idea?

El diseño perfecto depende del problema a resolver y, normalmente, el problema que queremos resolver cambia en el tiempo según las necesidades del usuario/empresa.

Diseño de software

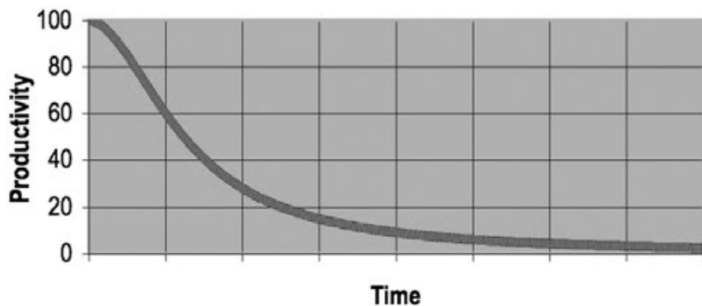
El diseño perfecto depende del problema a resolver y, normalmente, el problema que queremos resolver cambia en el tiempo según las necesidades del usuario/empresa.



Incluso si el problema no cambia (como en el proyecto) es muy difícil que nuestro diseño inicial sea bueno. Sobrepensar nuestro diseño inicial suele ser una mala idea.

Idea 2: Lancémonos a programar con la esperanza de que todo saldrá bien :)

Idea 2: Lancémonos a programar con la esperanza de que todo saldrá bien :)



Spoiler: Todo salió mal.

¿Pero entonces cómo lo hacemos para crear buen software?

¿Pero entonces cómo lo hacemos para crear buen software?

Emergence

Si sigues estas 4 reglas mientras desarrollas software, un buen diseño emergerá solo:

- 1 Pasa todos los tests.
- 2 No tengas código duplicado.
- 3 Que el código deje clara la intención del programador.
- 4 Reduce el número de clases y métodos.

¿Pero entonces cómo lo hacemos para crear buen software?

Emergence

Si sigues estas 4 reglas mientras desarrollas software, un buen diseño emergerá solo:

- 1 Pasa todos los tests.
- 2 No tengas código duplicado.
- 3 Que el código deje clara la intención del programador.
- 4 Reduce el número de clases y métodos.

Pregunta para ustedes:

¿Por qué seguir estas reglas nos podría ayudar a mejorar el diseño de nuestro software?

Regla 1: Pasa todos los tests.

Regla 1: Pasa todos los tests.

Lo más importante es que nuestro software funcione como debe. Hoy por hoy, la forma predilecta para verificar el funcionamiento de software es mediante tests automáticos.

¿Pero qué tienen que ver los tests con la calidad del diseño?

Regla 1: Pasa todos los tests.

Lo más importante es que nuestro software funcione como debe. Hoy por hoy, la forma predilecta para verificar el funcionamiento de software es mediante tests automáticos.

¿Pero qué tienen que ver los tests con la calidad del diseño?

Se dan tres argumentos principales:

- 1 Es más fácil testear clases pequeñas que hacen una sola cosa a la vez.
- 2 Clases que están muy acopladas son difíciles de testear.
- 3 Código testeado es más fácil de limpiar.

... por lo que agregar tests nos empujará hacia diseños que sigan buenos principios de diseño.

Regla 2: No tengas código duplicado.

Regla 2: No tengas código duplicado.

“Duplication is the primary enemy of a well-designed system. It represents additional work, additional risk, and additional unnecessary complexity.”

– Jeff Langr.

Regla 2: No tengas código duplicado.

“Duplication is the primary enemy of a well-designed system. It represents additional work, additional risk, and additional unnecessary complexity.”

– Jeff Langr.

“Cada vez que copies y pegues código piensa que probable hay algo mejor que podrías hacer.”

– Matías Recabarren.

```
2 public class VacationPolicy {  
3     public void accrueUSDivisionVacation() {  
4         // code to calculate vacation based on hours worked to date  
5         // ...  
6         // code to ensure vacation meets US minimums  
7         // ...  
8         // code to apply vacation to payroll record  
9         // ...  
10    }  
11    public void accrueEUDivisionVacation() {  
12        // code to calculate vacation based on hours worked to date  
13        // ...  
14        // code to ensure vacation meets EU minimums  
15        // ...  
16        // code to apply vacation to payroll record  
17        // ...  
18    }  
19 }
```

```
2 public abstract class VacationPolicy {
3     public void accrueVacation() {
4         calculateBaseVacationHours();
5         alterForLegalMinimums();
6         applyToPayroll();
7     }
8     private void calculateBaseVacationHours() { /* ... */ };
9     protected abstract void alterForLegalMinimums();
10    private void applyToPayroll() { /* ... */ };
11 }
12 public class USVacationPolicy: VacationPolicy {
13     protected override void alterForLegalMinimums()
14         { /* US specific logic */ }
15 }
16 public class EUVacationPolicy: VacationPolicy {
17     protected override void alterForLegalMinimums()
18         { /* EU specific logic */ }
19 }
```

Regla 3: Que el código deje clara la intención del programador.

Regla 3: Que el código deje clara la intención del programador.

El código se lee más de lo que se escribe. Por lo mismo, ser un profesional implica crear código que sea fácil de entender por el resto de tu equipo (incluyéndote a ti).

¿Cómo creo código que sea fácil de entender por otros?

Regla 3: Que el código deje clara la intención del programador.

El código se lee más de lo que se escribe. Por lo mismo, ser un profesional implica crear código que sea fácil de entender por el resto de tu equipo (incluyéndote a ti).

¿Cómo creo código que sea fácil de entender por otros?

- Usa buenos nombres. El nombre de una clase o método es bueno si al leer su código no te sorprende su contenido.
- Que el código de tus clases y funciones sean cortos.
- Que tus test unitarios sean fáciles de entender (sirven como documentación).
- ...

Regla 3: Que el código deje clara la intención del programador.

El código se lee más de lo que se escribe. Por lo mismo, ser un profesional implica crear código que sea fácil de entender por el resto de tu equipo (incluyéndote a ti).

¿Cómo creo código que sea fácil de entender por otros?

- Usa buenos nombres. El nombre de una clase o método es bueno si al leer su código no te sorprende su contenido.
- Que el código de tus clases y funciones sean cortos.
- Que tus test unitarios sean fáciles de entender (sirven como documentación).
- ...

Pero la regla más importante es que lo **intentas**.

Regla 4: Reduce el número de clases y métodos.

Regla 4: Reduce el número de clases y métodos.

Esta es la regla con **menos prioridad** pero su rol es recordarnos que no hay que obsesionarse con las buenas prácticas y limpieza de nuestros códigos.

Regla 4: Reduce el número de clases y métodos.

Esta es la regla con **menos prioridad** pero su rol es recordarnos que no hay que obsesionarse con las buenas prácticas y limpieza de nuestros códigos.

A veces, nos encontramos que software cuyo alto número de clases y métodos se debe simplemente a dogmatismos sin sentido.

Regla 4: Reduce el número de clases y métodos.

Esta es la regla con **menos prioridad** pero su rol es recordarnos que no hay que obsesionarse con las buenas prácticas y limpieza de nuestros códigos.

A veces, nos encontramos que software cuyo alto número de clases y métodos se debe simplemente a dogmatismos sin sentido. E.j.,

- Forzar a que se defina una interfaz por cada clase en tu programa.
- Forzar a que se separen datos de comportamiento en clases distintas.
- ...

Regla 4: Reduce el número de clases y métodos.

Esta es la regla con **menos prioridad** pero su rol es recordarnos que no hay que obsesionarse con las buenas prácticas y limpieza de nuestros códigos.

A veces, nos encontramos que software cuyo alto número de clases y métodos se debe simplemente a dogmatismos sin sentido. E.j.,

- Forzar a que se defina una interfaz por cada clase en tu programa.
- Forzar a que se separen datos de comportamiento en clases distintas.
- ...

Entonces, es bueno minimizar el número de clases y métodos en la medida de lo posible... pero no al costo de romper las reglas 1, 2 y 3.

Resumen

Si sigues estas 4 reglas mientras desarrollas software, un buen diseño emergerá solo:

- 1 Pasa todos los tests.
- 2 No tengas código duplicado.
- 3 Que el código deje clara la intención del programador.
- 4 Reduce el número de clases y métodos.

Resumen

Si sigues estas 4 reglas mientras desarrollas software, un buen diseño emergerá solo:

- 1 Pasa todos los tests.
- 2 No tengas código duplicado.
- 3 Que el código deje clara la intención del programador.
- 4 Reduce el número de clases y métodos.

Espera, estoy confundido.



Espera, estoy confundido.



¿En el curso de diseño me estás recomendando que no haga un diseño?

Capítulo 11: Systems by Dr. Kevin Dean Wampler

Capítulo 11: Systems by Dr. Kevin Dean Wampler

It is not necessary to do a Big Design Up Front (BDUF). In fact, BDUF is even harmful because it inhibits adapting to change, due to the psychological resistance to discarding prior effort and because of the way architecture choices influence subsequent thinking about the design.

Capítulo 11: Systems by Dr. Kevin Dean Wampler

It is not necessary to do a Big Design Up Front (BDUF). In fact, BDUF is even harmful because it inhibits adapting to change, due to the psychological resistance to discarding prior effort and because of the way architecture choices influence subsequent thinking about the design.

Building architects have to do BDUF because it is not feasible to make radical architectural changes to a large physical structure once construction is well underway. Although software has its own physics, it is economically feasible to make radical change, if the structure of the software separates its concerns effectively.

Capítulo 11: Systems by Dr. Kevin Dean Wampler

It is not necessary to do a Big Design Up Front (BDUF). In fact, BDUF is even harmful because it inhibits adapting to change, due to the psychological resistance to discarding prior effort and because of the way architecture choices influence subsequent thinking about the design.

*Building architects have to do BDUF because it is not feasible to make radical architectural changes to a large physical structure once construction is well underway. Although software has its own physics, it is economically feasible to make radical change, **if the structure of the software separates its concerns effectively.***

Capítulo 11: Systems by Dr. Kevin Dean Wampler

It is not necessary to do a Big Design Up Front (BDUF). In fact, BDUF is even harmful because it inhibits adapting to change, due to the psychological resistance to discarding prior effort and because of the way architecture choices influence subsequent thinking about the design.

*Building architects have to do BDUF because it is not feasible to make radical architectural changes to a large physical structure once construction is well underway. Although software has its own physics, it is economically feasible to make radical change, **if the structure of the software separates its concerns effectively.***

This means we can start a software project with a “naively simple” but nicely decoupled architecture, delivering working user stories quickly, then adding more infrastructure as we scale up.

El extracto clave es el siguiente:

“Although software has its own physics, it is economically feasible to make radical change, if the structure of the software separates its concerns effectively.”

El extracto clave es el siguiente:

“Although software has its own physics, it is economically feasible to make radical change, if the structure of the software separates its concerns effectively.”

¿Cuál es una buena estructura base para comenzar?

Modelo-Vista-Controlador